全栈开发技术期末项目报告

阅记

阅读与书摘管理应用

项目名称：阅记阅读与书摘管理应用

姓名：周宇斌

学号：20246085

班级：软件 2401

2026 年 10 月 20 日

本文依据实际源码、自动化测试及 Linux 桌面实测编写，包含本轮缺陷修复与回归验证。测试环境和结果均如实记录；Windows 桌面尚未实测，不记为已通过。

目录

项目背景与意义 **2**

1 需求分析 **2**

1.1 功能需求 2

1.2 非功能需求 3

2 系统设计 **3**

2.1 架构设计 3

2.2 功能模块设计 4

2.3 数据/数据库设计 4

2.4 接口与类设计 4

2.5 技术选型说明 5

3 系统实现 **5**

3.1 关键模块的核心实现 5

3.2 核心功能与 UI 截图 8

3.3 难点和解决方案 13

4 系统测试 **13**

4.1 测试环境 13

4.2 测试用例 14

4.3 测试截图 15

5 总结与展望 **18**

项目背景与意义

个人阅读中，书籍资料、阅读活动与摘录通常分散记录。阅记以本地离线应用统一管理这三类信息，并由阅读记录推导当前进度、计划状态和统计结果，使用户能够追踪阅读活动、整理感想及恢复自己的数据。

项目不提供电子书阅读器、账号登录、联网检索或云同步。其业务边界是“记录阅读”，用户录入的阅读记录表示自身活动声明，不能据此推断所有未记录页段均已读过。演示数据中的书名、作者、摘录和感想均明确标注为虚构资料。

1 需求分析

1.1 功能需求

应用实现录入书籍、制定计划、记录阅读、整理书摘和查看统计的完整闭环。十个独立业务页面及主要能力如下。

页面	已实现能力
阅读概览	在读书籍、近期记录、待完成计划、全量累计指标、书库与详情入口。
我的书库	书名或作者搜索、未开始/在读/已完成筛选、每页 10 本分页、新增及详情入口。
书籍详情	资料、当前进度、累计活动、记录增改删弹窗、关联书摘与计划、级联删除确认。
书籍编辑	书名、作者、分类、总页数与简介编辑，校验已有最大引用页码。
书摘列表	正文或书名搜索、收藏筛选、编辑、收藏切换和删除。
书摘编辑	选择书籍、编辑摘录与感想、可选页码和收藏状态。
阅读计划	查看进行中、已完成与已逾期计划，编辑和删除。
计划编辑	选择书籍、设定目标日期、根据剩余页数和天数展示建议。
阅读统计	累计页数、分钟数、完成书籍数与近七日阅读趋势。
设置与数据	浅深主题、空库载入虚构演示数据、JSON 导出与带自动备份的恢复。

初始化页、主窗口、主容器和阅读记录弹窗不用于补足十个业务页面。编辑未保存时，返回、切换菜单与关闭窗口共用保存、放弃、取消离开语义；保存失败保留草稿。列表返回刷新数据，保留当前搜索条件。

1.2 非功能需求

业务数据保存在用户本地应用数据目录，断网后核心业务可用。首次依赖还原需要 NuGet 来源；“运行离线”不等于“首次还原离线”。数据库写入串行，跨表修改采用事务。异步命令处理 IO、忙碌状态和错误重试。

界面使用中文标签、侧边导航、统一表单及浅深主题。原生文件选择器使用系统语言，因此测试环境的 GTK 选择器部分文字为英文。代码采用 UI 与 Library 职责分离，业务服务通过接口注入，便于替换依赖测试。本文不提供未经测量的性能指标，也不宣称已完成大规模数据压力测试。

2 系统设计

2.1 架构设计

解决方案包含 ReadJournal、ReadJournal.Library 及 ReadJournal.UnitTest 三个工程。UI 负责视图、应用资源、定位器、导航实现、文件选择和提示；Library 负责模型、规则、存储、统计、备份及 ViewModel，不引用 Avalonia 控件；测试工程使用 xUnit、Moq 和临时真实 SQLite 数据库。

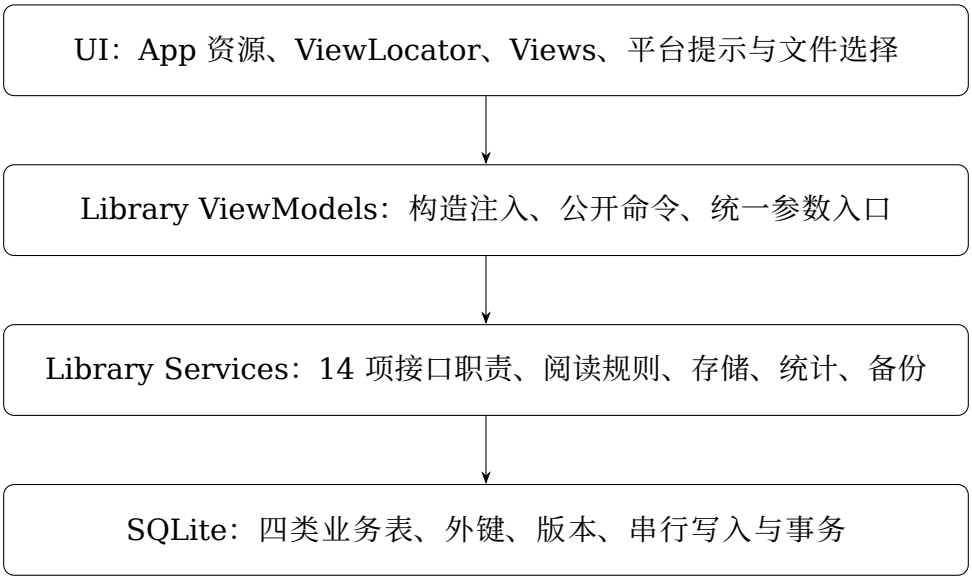


图 1：依据实际工程组织绘制的架构图

保留 DailyPoetryA 的完整组装机制：ServiceLocator 构造内局部 ServiceCollection 注册、BuildServiceProvider 构建、只读 IServiceProvider 保存、属性 GetService 解析；Current 通过 Application 资源读取并缓存。全部服务和 ViewModel 为单例，构造函数只保存依赖并建立命令，不启动导航或加载。

根导航替换 MainWindowViewModel.Content；菜单导航通过 SetMenuAndContent 清空返回栈并设置菜单；内容导航调用 PushContent 压栈。返回栈及菜单状态属于 MainViewModel，未增加导航宿主或页面工厂。ViewLocator 实现 IDataTemplate，以命名替换、反射和 Activator 创建视图，按类型缓存并设置 DataContext。

2.2 功能模块设计

菜单提供概览、书库、书摘、计划、统计及设置六个入口。书库进入详情或书籍编辑；详情进入书籍编辑、书摘编辑、计划编辑及记录弹窗；书摘和计划列表进入相应编辑页。内容返回保留上一级业务上下文，菜单切换清空历史栈。

导航顺序为定位目标单例、SetParameter、切换内容或压栈、Loaded 公开异步命令加载。加载失败保留目标页面，呈现错误及重试入口。重复同一单例内容目标在 SetParameter 之前拦截，避免覆盖栈中原参数；不同书籍经返回书库再打开，不构造同类型详情的多份历史快照。

2.3 数据/数据库设计

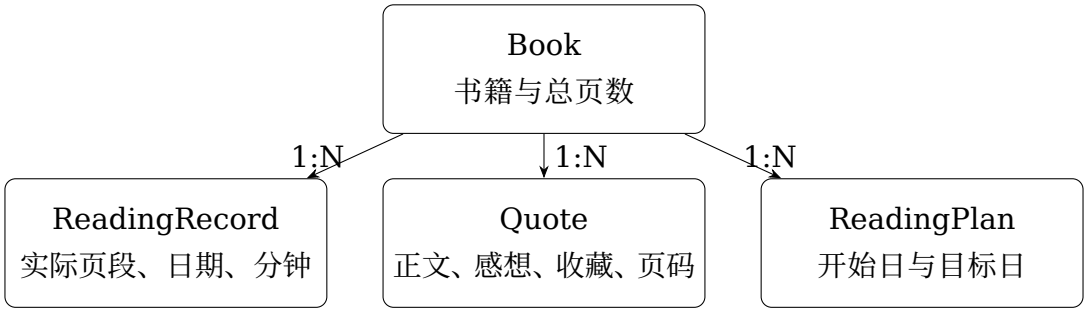


图 2: 实际四表实体关系；子表 BookId 外键级联删除

Book 含 Id、Title、Author、Category、TotalPages、Description、CreatedAt 和 UpdatedAt。ReadingRecord 含 Id、BookId、ReadingDate、StartPage、EndPage 和 Minutes。Quote 含 Id、BookId、Text、Thoughts、可空 Page、IsFavorite 及创建/修改时间。ReadingPlan 含 Id、BookId、StartDate 和 TargetDate。数据库 user_version 为 1，初始化事务创建表并启用外键。

记录约束为 1 ≤ 起始页 ≤ 结束页 ≤ 总页数、分钟数为正整数、日期不晚于本地当天。当前页取最大结束页，无记录为 0；累计页数对每条记录求闭区间长度之和，包含重读。完成日期为读至末页记录的最早日期。修改记录或总页数后由有效数据重新推导状态。

新建或编辑计划时，同一本书不能再关联第二个计划；普通新计划不能关联已完成书籍。旧备份中的多个完成计划可保留，但修改记录或总页数会使其同时变为未完成时，整个操作拒绝并提示先清理多余计划。未逾期的建议每日页数为剩余页数除以包含当天的剩余天数后向上取整；完成为 0；逾期单列状态与剩余页数。删除计划不删除记录。

2.4 接口与类设计

最终采用与参考相同的语法树口径：只扫描 UI 与 Library 手写 class/interface，包括默认访问级别；partial 按工程及完整类型名去重；排除测试和生成代码，不计枚举、结构体或 record。实际为 66 类、14 接口、77 个 C# 源文件、14 个视图文件，其中业务视图 10 个。参考为 63 类、10 接口、9 视图；最低 45 类、7 接口，实际数量已满足。

14 项接口职责为数据库初始化、书籍存储、阅读记录存储、书摘存储、计划存储、偏好、阅读业务、统计、备份、提示、文件选择及三层导航。职责与实现逐一对应，未通过空接口或人为拆分补足数量。主要类还包括四类实体、进度/计划/统计投影、共享编辑基类、各页 ViewModel、平台视图和定位器。新增计划一致性规则类在事务中复用业务状态校验，避免不同写入路径与备份规则分歧。

完整逐类型名称、路径、行号及源文件哈希见`docs/application-type-inventory.json`和同名 Markdown。参考复核结果保留于`docs/reference-baseline-cloud.json`，可用`docs/tools/TypeInventory` 重跑；原 PowerShell 脚本仍保留。

2.5 技术选型说明

技术	实际版本和用途
.NET / C#	net8.0，SDK 8.0.424 固定，跨平台桌面业务。
Avalonia	11.0.10；Desktop、Fluent、Inter 同版本；行为包 11.0.10.9。
CommunityToolkit.Mvvm	8.2.2；通知属性、RelayCommand 及 AsyncRelayCommand。
sqlite-net-pcl	1.8.116；本地 SQLite 及事务。
Microsoft.Extensions.DependencyInjection	8.0.0；真实 IoC 容器和构造注入。
xUnit / Moq	2.4.2 / 4.20.70；测试 SDK17.6.0, Runner2.4.5。
Avalonia.Headless	11.0.10，仅用于自动化中的真实应用资源和视图机制测试。

版本遵循参考项目，不以升级替代原有架构。真实桌面验证另在 Xvfb 上启动 Avalonia 应用，未将 Headless 断言冒充桌面操作。首次完整构建有 3 条 NETSDK1206 提示，源于固定 SQLite 原生依赖含旧 Alpine RID；Linux x64 实测正常，不据此宣称支持 Alpine。

3 系统实现

3.1 关键模块的核心实现

以下摘录直接来自交付源码。定位器构造中的注册均使用 AddSingleton；末尾构建真实容器，属性由容器解析。

```
public ServiceLocator()
{
    var serviceCollection = new ServiceCollection();
```

```
        serviceCollection.AddSingleton<DataPaths>();
        serviceCollection.AddSingleton<IDatabaseService, DatabaseService>();
        serviceCollection.AddSingleton<IRootNavigationService, RootNavigationService>();
        serviceCollection.AddSingleton<IMenuNavigationService, MenuNavigationService>();
        serviceCollection.AddSingleton<IContentNavigationService, ContentNavigationService>();
        serviceCollection.AddSingleton<MainWindowViewModel>();
        serviceCollection.AddSingleton<InitializationViewModel>();
        serviceCollection.AddSingleton<MainViewModel>();
        serviceCollection.AddSingleton<OverviewViewModel>();
        serviceCollection.AddSingleton<BookListViewModel>();
        serviceCollection.AddSingleton<BookDetailViewModel>();
        serviceCollection.AddSingleton<IReadingService, ReadingService>();
        serviceCollection.AddSingleton<IBookStorage, BookStorage>();

        serviceCollection.AddSingleton<ReadingPlanListViewModel>();
        serviceCollection.AddSingleton<ReadingPlanEditViewModel>();
        serviceCollection.AddSingleton<StatisticsViewModel>();
        serviceCollection.AddSingleton<SettingsViewModel>();
        _serviceProvider = serviceCollection.BuildServiceProvider();
    }
    public MainWindowViewModel MainWindowViewModel =>
    ↪  _serviceProvider.GetService<MainWindowViewModel>()!;
    public InitializationViewModel InitializationViewModel =>
    ↪  _serviceProvider.GetService<InitializationViewModel>()!;
    public MainViewModel MainViewModel => _serviceProvider.GetService<MainViewModel>()!;
    public OverviewViewModel OverviewViewModel =>
    ↪  _serviceProvider.GetService<OverviewViewModel>()!;
```

内容导航保留常量 switch 映射与定位器调用。在阅记新增的重复单例保护之后统一注入参数并压栈，未引入其他导航机制。

```
    public void NavigateTo(string view, object? parameter = null)
    {
        ViewModelBase target = view switch
        {
            ContentNavigationConstant.BookDetailView =>
            ↪  ServiceLocator.Current.BookDetailViewModel,
            ContentNavigationConstant.BookEditView => ServiceLocator.Current.BookEditViewModel,
            ContentNavigationConstant.QuoteEditView =>
            ↪  ServiceLocator.Current.QuoteEditViewModel,
            ContentNavigationConstant.ReadingPlanEditView =>
            ↪  ServiceLocator.Current.ReadingPlanEditViewModel,
            _ => throw new ArgumentException(" 未知或尚未实现的内容目标。", nameof(view))
        };
        // 阅记新增保护：必须在 SetParameter 之前拦截重复单例，避免覆盖原参数。
        if (ServiceLocator.Current.MainViewModel.ContentStack.Contains(target)) return;
        target.SetParameter(parameter!);
        ServiceLocator.Current.MainViewModel.PushContent(target);
    }
}
```

阅读业务通过有效记录计算当前进度、累计活动和完成日期，重读不会叠加当前进度。

```
    }
    public BookProgress Calculate(Book book, IEnumerable<ReadingRecord> records)
    {
        var all = records.Where(x => x.BookId == book.Id).ToArray();
```

```
        return new BookProgress
        {
            Book = book,
            CurrentPage = all.Select(x => x.EndPage).DefaultIfEmpty(0).Max(),
            TotalReadPages = all.Sum(x => (long)x.EndPage - x.StartPage + 1),
            TotalMinutes = all.Sum(x => (long)x.Minutes),
            CompletedOn = all.Where(x => x.EndPage == book.TotalPages).Select(x =>
            ↪  (DateTime?)x.ReadingDate.Date).Min()
        };
    }
}
```

备份由格式版本和四类集合组成，每个持久化字段要求显式存在。恢复先验证完整文档，在串行事务边界捕获原快照并成功写出自动备份，再删除并恢复业务表。自动备份失败不替换，写入失败事务回滚，成功后用户再进入页面会重新加载；主题在独立偏好文件中。

3.2 核心功能与 UI 截图

以下十图均为最终 Release 应用在 Linux 真实 X11 窗口中的直接截屏，显示的是用户主动载入的虚构演示资料。非全部内容都能在同一窗口高度内展示时，页面提供滚动；截图不拼接或补画隐藏内容。



图 3: 阅读概览：全量指标、在读书籍和近期记录



图 4: 我的书库：搜索、状态筛选、分页与详情入口



图 5：书籍详情：进度、阅读活动与计划关联



图 6：书籍编辑：独立业务表单



图 7：书摘列表：虚构摘录、感想和收藏入口



图 8：书摘编辑：正文、感想、页码和收藏



图 9：计划列表：派生状态与建议页数



图 10：计划编辑：实际关联书籍与目标日期



图 11：统计：191 页、210 分钟、完成 1 本及近七日趋势



图 12：设置与数据：主题、导出、恢复和演示入口

3.3 难点和解决方案

实际问题	已执行处理及验证
参考源码最初不可用	Gitee 代理 CONNECT403，本地无参考目录；用户上传后同步副本，64 文件哈希及语法树类型清单与原基线完全一致，才进入骨架开发。
命令绑定和目标错误	实际参考 InitializationViewModel.cs:25 命令为 private，视图通过公开绑定访问；TodayViewModel.cs:76 用内容服务传菜单目标。阅记公开命令、按层使用服务与常量，参考副本保持只读。
XAML 与 SQLite 首轮失败	内部 xmlns 导致 AXN0002，移至根节点；busy_timeout 查询返回行，改用 ExecuteScalar。修复后重跑当前阶段构建及测试。
单例异步旧数据	引入参数版本校验，旧请求不发布到新参数页面；加载命令发现版本变化继续加载新参数。延迟任务用例验证 A 不会覆盖 B。
多次离开确认	主 ViewModel 共用防重入保护；返回、菜单、关闭不会并发打开多个确认窗口。保存失败保留草稿。
绑定诊断	事件命令显式绑定同一资源定位器中的单例，解决构造时 DataContext 未就绪；进度条使用静态模板，避免固定 Fluent 版本动画绑定。最终遍历十页绑定诊断为 0。
原生选择器驱动	GTK 路径自动补全曾干扰测试输入，应用正确提示文件不存在。通过真实列表选择同一文件后恢复成功，未用后台写库代替 UI 恢复。

本轮复核还发现加载与保存期间输入保护不足，以及计划状态变化与备份校验不一致。修复后，表单按字段合并未改值并保留用户修改，保存仅确认实际提交的快照；保存中新增修改保留在编辑页，新建成功回填主键避免重复插入。加载失败后可在编辑页重新加载。计划变更、记录修改与总页数修改在同一数据库事务中维护约束，拒绝不安全操作且不删除原数据。

重复目标拦截、参数版本、草稿保护、串行写入和事务均为阅记新增业务正确性措施。参考 README 明确未解决线程与原子性，不能把这些新增设计称为参考项目原有机制。

4 系统测试

4.1 测试环境

实测为 Debian13 / Linux x64，.NET SDK8.0.424、运行时 8.0.30、MSBuild17.11.48，Xvfb21.1.16、1440×1000 显示环境。自动化运行 Release 构建；业务测试使用独立临时 SQLite

库，架构测试使用真实容器和应用资源。Windows 桌面验收未执行。

真实图形环境中的窗口无需联网即可使用业务功能。图形日志包含 X11 会话管理器缺失及 GLX llvmpipe 回退提示，应用实际继续运行并完成操作；该记录不代表已测 GPU 硬件加速。最终十页面运行的绑定诊断为 0，完整日志保留。

最终源码归档按显式白名单生成，共 114 个文件、102816 字节，无 bin、obj、TestResults、运行数据库或工作过程目录。解压到新临时目录，使用全新的 NuGet 包及 HTTP 缓存目录执行：

```
dotnet restore ReadJournal.sln --locked-mode --no-cache
dotnet build ReadJournal.sln -c Release --no-restore
dotnet test ReadJournal.sln -c Release --no-build --no-restore
```

还原、构建、测试均退出 0；68 项测试通过。依赖还原实际使用网络，未宣称离线首次还原。解压后的实际应用分别首次、再次启动成功，并在独立运行目录创建版本 1 数据库。构建结果不回填源码包。

4.2 测试用例

用例组与数量	实际覆盖与结果	结果
架构 5	容器类型与全部 14 接口单例、构造注入、资源 Current、三层参数与返回栈、反射视图缓存、失败重试、A/B 详情更新。	通过
数据库 3	初始化幂等与重启持久化、真实外键失败回滚、目录创建失败移除原因后重试。	通过
阅读业务 12	第一页/单页/末页/整本；10～20 计 11 页及重复 22 页；0 页、逆序、超限、非正分钟、未来日期拒绝；进度重算、搜索分页、收藏、级联。	通过
编辑 2	取消、放弃、保存并离开；模拟保存失败保留草稿和原页面；成功保存后不再脏。	通过
扩展业务 16	计划当天/逾期/完成分母边界；唯一未完成计划；统计及演示；非连续 ID 往返；无效备份九类；自动备份失败和事务替换失败回滚；偏好保留。	通过
时序 2	延迟旧详情加载不能发布到新参数；返回、菜单与关闭不会同时发起两次确认。	通过
编辑回归 20	首次加载前和加载中逐字段草稿保留、失败后保存门禁与重试；四种编辑器保存及保存离开期间新输入留页、新建 ID 回填。	通过

计划回归 8	重复关联提前拒绝、历史多完成计划的删除/改页/换书/增总页安全拒绝、无部分写入、清理后正常操作、安全编辑及并发唯一性、JSON 往返。	通过

本轮真实 UI 复验中，对隔离测试库施加独占锁触发读取失败；界面禁止将未水合的空表单保存回库，解除锁并重新加载后资料完整恢复。编辑计划关联到已有计划的完成书籍时，错误明确提示且原关联保持不变；随后通过原生文件选择器导出和恢复 3 本书、4 条记录、2 条书摘、3 个计划，自动备份存在。

单条用例名称、参数、耗时和结果见`docs/test-results.md`，来源为真实 fix-release.trx。源码包解压目录同一测试集再次 68 项通过。UI 另行实际验证书籍录入、阅读记录与书摘、未保存取消/放弃、重启持久化、主题、演示、计划状态与建议、原生备份恢复；这些证据不是由自动化结果推定。补充一致性实测在隔离演示库副本中仅用 UI 修改末页记录至 240 页，统计更新为 371 页、210 分钟、完成 2 本，关联计划完成；删除该完成记录后统计为 161 页、175 分钟、完成 1 本，进度与计划退回在读；再通过书籍编辑把总页数改为 30 页，完成书数回到 2 本且累计活动仍为 161 页、175 分钟。

4.3 测试截图

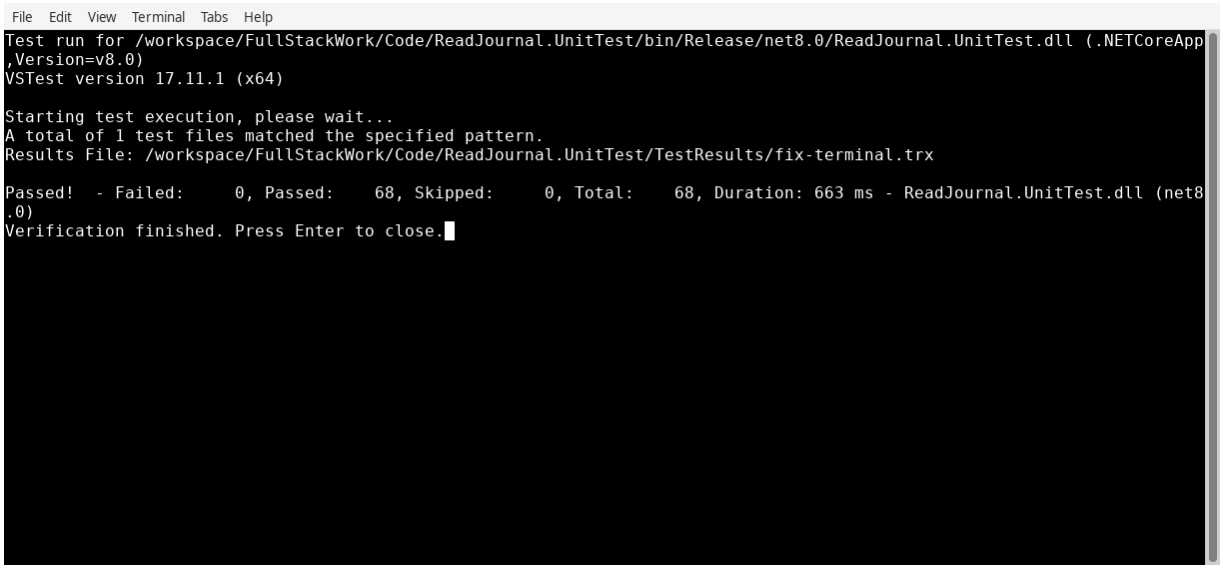


图 13: 真实终端执行 Release 测试：68 通过、0 失败、0 跳过

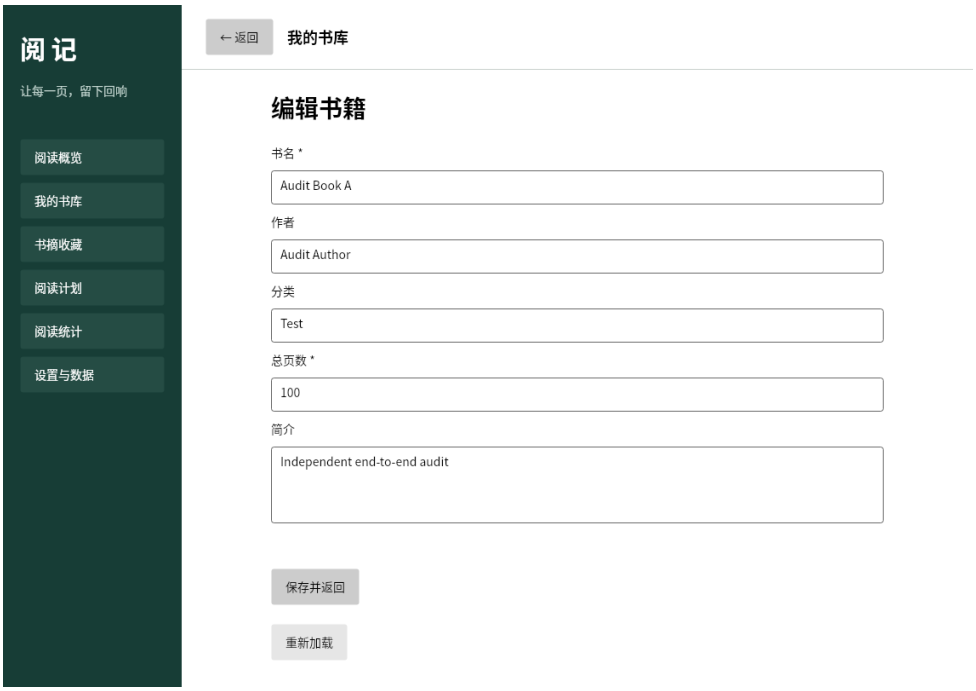


图 14：编辑页读取失败后，通过重新加载恢复原资料



图 15: 重复关联至已有计划的书籍时拒绝保存，原关联保持不变



图 16: 最终源码包解压、还原、构建后在独立数据目录实际首次启动

5 总结与展望

阅记完成十个业务页面及阅读管理闭环，保留参考项目真实 IoC、单例定位器、三层导航、统一参数入口、主 ViewModel 返回栈和反射视图缓存。68 项自动化测试及 Linux 实际桌面流程通过；源码包满足白名单和大小约束，解压后可还原、构建、测试及启动。

报告身份资料已补齐，当前交付包含修复后的源码与正式 PDF。规划默认的 Windows 桌面尚未实际验收，不能以 Linux 结果推定 Windows 通过。初次构建存在固定 SQLite 依赖的 Alpine RID 提示，未执行 Alpine 兼容、大数据压力或多设备同步测试。未来可在不破坏现有调用机制的前提下补充更多输入可用性与更大数据集测量；电子书阅读、联网检索和云同步属于第一版边界之外。

本报告依据原始指导书模板、唯一开发规划、实际参考副本和最终实现编写。相关证据分别保留于来源记录、参考架构核对、开发执行记录、类型清单、测试结果和真实截图目录；参考源码及 raw 原始资料未改写。